Data Integrity - Patterns & Anti-Patterns

> **Note:** This Support Document has been migrated from the Pega Community Wiki.

*Authored by **Vinod Pakalapati***

Data Integrity is a quintessential part of any enterprise application design. With multiple requestors accessing and updating data within an application, it is imperative to make design choices which eliminate chances of data integrity issues creeping in. In this article, the emphasis will be on detailing patterns and anti-patterns relating to the use of the *Obj-Save* method, the *Obj-Save with write now* method and the use of explicit *Commit* methods in an activity. The *Save-DataPage* method also falls in the same category, but its use has not been detailed in this article.

Pega's out-of-the-box flow processing takes care of required commits, whenever a flow ends, whenever an assignment is created or completed, etc. To avoid interference with the default transaction model provided by Pega Platform™, it's recommended not to use explicit *Obj-Save with write now* or *Commit* methods in flow-related rules, like utilities. Explicit use of *Obj-Save with write now* or *Commit* methods can cause data integrity issues. Flow processing already manages the boundaries of work-related database transactions, and adding explicit commits can create synchronization problems between what is assumed to be in the database and what is actually in the database. This situation can make it extremely difficult to track bugs and can result in stale database records.

Pega Platform default behavior for performing *Obj-Save* is to defer the actual UPDATE/INSERT SQL command until the system performs an automatic commit. As a result, you can perform multiple back-to-back saves on the same object instance and the Pega Platform engine will combine these updates into a single cumulative update at the time of the commit. This maximizes concurrency within the application. Using *Obj-Save with write now* enabled or *Commit* methods in an activity interferes with the default Pega Platform behavior.

There could be legitimate reasons for performing explicit *Commit* or *Obj-Save with write now* methods, such as a need to save objects (non-case objects) through custom activities or cases through activities handling bulk processing. But using these methods can be dangerous if proper locking and error handling is not done. As this is not a common, recommended practice, appropriate warnings are shown on the rule form whenever these methods are used. It is strongly advised to pay heed to the related guardrails.

Using Obj-Save

The *Obj-Save* method is used to save a clipboard page to the PegaRULES database or to an external database if the page belongs to an external class. The method uses properties on the page to derive the internal key under which it will be saved. This method can create a new instance or overwrite a previous instance with that key.

It's recommended not to use an explicit *Obj-Save with write now* method as it can lead to data integrity issues.

Using *Obj-Save* skips common processing such as validation, creating history, and keeping the properties that track last information update. We need to programmatically update properties such as *pxUpdateDateTime*, *pxCreateDateTime*, *pxUpdateOperator*, *pxCreateOperator* etc.

In most situations it is recommended to rely on flow processing or activity APIs such as ***UpdateWorkObject*** and ***Save*** instead of using the *Obj-Save* method. The APIs would update properties such as *pxUpdateDateTime, pxCreateDateTime, pxUpdateOperator, pxCreateOperator* etc.

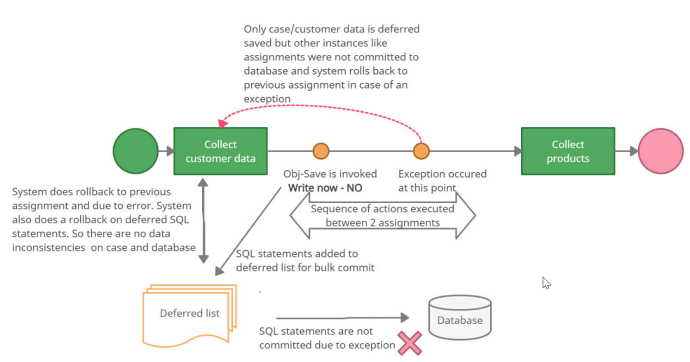
Obj-Save without write now

Using the *Obj-Save without write now* method will not commit the data immediately to the database but add the update/create query to a deferred list. The data is committed when the system does a typical commit as part of case processing.

When the *Obj-Save* method is executed, only the cases or data objects on which the method is invoked, are saved; but the rest of the instances, such as assignments, are still not committed until the system does the commit as part of case processing. So, in case of any exceptions in the case context after invoking the *Obj-Save* method and before the system commits the change, the system rolls back to the previous state.

Let's consider a use case where a user collects customer data and products. In between collecting the customer data and products, certain actions, like saving customer data to the database and sending an email, are performed.

- After the *Collect customer data* step, a utility (refer to the first orange shape in the pic below) is invoked where *Obj-Save without write now* is used on the customer data. The SQL statement is added to a deferred list but not committed to the database. The queries in the deferred list will be committed to the database later whenever the system does the commit.
- In a happy path scenario, if there are no exceptions in the processing after calling the *Obj-Save* method, the flow processing moves to the *Collect products* assignment and commits the changes. As part of the system commit, the customer details are persisted to the database.
- In case of any exceptions occurring as part of the case processing after invoking the *Obj-Save* method (refer to the second orange shape in the pic below), the system restores the previous assignment (such as the *Collect customer data* step) and reverts any uncommitted data. In this scenario, the customer data is not persisted to the database.



Pega provides design choices for rolling back: either the most recent only, or the complete set of deferred list statements. For example:

- Usage of the *Obj-Save-Cancel* method cancels the most recent uncommitted *Obj-Save* method, so that the instance is not written as part of a later *Commit* operation. We can also use this method to undo an *Obj-Delete* method that has not yet been committed.
- Usage of the *Rollback* method cancels all uncommitted *Obj-Save* and *Obj-Delete* methods that are in the deferred list, not only the most recent one.

Obj-Save with write now

Using *Obj-save with write now* method commits the data immediately to the database without waiting for the default system commit.

When an *Obj-Save with write now* method is executed, only the cases or data objects on which the method is invoked, are committed, but the rest of the instances, such as assignments, are not. So, in case of any exceptions in the case context, after invoking the *Obj-Save with write now* method, the committed changes are not rolled back, but the system restores the rest of the data to the previous state. Due to this, data inconsistencies may occur.

In the scenario considered earlier, in this case, the customer data subjected to *Obj-Save with write now* gets persisted immediately. On an exception, the system restores the case to the previous assignment (*Collect customer data)*, but the customer data is already persisted to the database. Due to this, the customer data in the clipboard memory is different from what would have been persisted to the database.

Obj-Save with write now method

Using the explicit Commit method

The *Commit* method in an activity is used to persist all uncommitted changes to the database. This method writes all the instances (deferred list) specified by one or more earlier *Obj-Save* methods to the PegaRULES database for internal classes, or to external databases, for external classes. If any of the write operations fail, all instances in the deferred list fail and are not committed.

Loss of assignment due to an explicit commit

It is recommended not to use the explicit *Commit* method as it can lead to data integrity and locking issues.

For example, consider a scenario where an explicit *Commit* method was used in the case processing and after the *Commit* method, an exception occurred. When the *Commit* method was issued, all deferred operations were committed to the database immediately. Also, the previous assignment was completed, and changes were persisted to the database. Now, when the exception occurs after the commit, the system cannot rollback to the previous assignment as it does not exist anymore. For this reason, case processing stops with a processing error. The following image explains this scenario using the earlier example.

Commit

In case of any legitimate reasons for performing an explicit commit, design the processing so that the *Commit* method occurs only after errors are intercepted, checked, and corrected. If the *Commit* method fails, it is unlikely that the processing can continue, so it is important to include a transition in the *Commit* step that checks the status and handles errors and exceptions accordingly.

Locking issues due to explicit commit

Locking on the Pega Platform side is different from the typical database locking mechanism supplied by the database software, such as Oracle. Pega locks are exclusive to one thread and operate system-wide in a multi-node system.

Pega locks are acquired by using different APIs provided by the Pega Platform. For example, using one of the three methods *Obj-Open*, *Obj-Open-by-Handle,* or *Obj-Refresh-and-Lock* in an activity might acquire a Pega lock. These methods also have a **Release on commit** option to release the lock when the next commit command is encountered.

A commit operation releases locks that are held by a requestor session, except for objects opened without selecting the **Release on commit** check box.

After a commit operation by one requestor, for example requestor A, locks are released and a different requestor, for example requestor B, can acquire a lock on an instance if required. But in a case when the requestor A wants to continue with the current session, it may not be able to do so as the lock might have been ac-

are as follows:

- When a work item is created.
- When processing for a submit operation of a flow action completes, for both connector and local flow actions.
- When *GetNextWork* processing transfers an assignment from a work queue to a worklist.
- When a work item is updated, resolved, or reopened by a flow action.
- When additional flow execution starts for an existing work item.

To prevent inadvertent loss of locks, activities in an application that execute within a flow should not execute the *Commit* method. In case of any legitimate reasons for performing an explicit commit in an activity, after the *Commit* method, check the availability of the lock and then reacquire any needed locks, if not present.

Related information

For more information, refer to the following articles on Pega Community:

Page	URL	Description
Pega Community	**https://community.pega.com/knowledgebase/articles/reference/86/obj-save-method**	*Obj-Save* method
Pega Community	**https://community.pega.com/knowledgebase/articles/reference/86/commit-method**	*Commit* method
Pega Community	**https://community.pega.com/knowledgebase/articles/case-management/86/managing-concurrent-access-case**	Pega locking mechanism

Pega Platform 7.1.1 **Pega Platform** **Enterprise Application Development** **Design Patterns**